

Brief para documentación

Mercado Secundario IFC sobre Avalanche — Hackathon Avalanche LATAM 2026

*Este documento es un **inventario**. Lista qué hicimos, con qué, y por qué importa, sin entrar en detalle. La persona que documente debe expandir cada ítem explicando **por qué se eligió** y **para qué sirve** en el contexto del proyecto.*

0. Sobre el proyecto

Plataforma de **tokenización y mercado secundario regulado** para participaciones de Instituciones de Financiamiento Colectivo (IFC) en México, sobre Avalanche, con compliance CNBV embebido a nivel de smart contract. Cliente piloto: Arkangeles.

- Repo: <https://github.com/Kaii35/hackathon-avalanche>
 - Stack: monorepo pnpm + Turborepo (apps/web, apps/indexer, packages/contracts, packages/sdk, packages/shared, packages/ui, packages/database, packages/config)
-

1. Problemas que resolvemos

- Illiquidez de inversionistas IFC (5–10 años atrapados sin poder vender)
 - Cap table manual en Excel y reconciliación entre fundadores/IFC/inversionistas
 - KYC repetido cada vez que un inversionista entra a una nueva oferta o plataforma
 - Fricción operativa para distribuir dividendos a cientos de holders
 - Compliance no auditable: la IFC firma cumplimiento pero no hay rastro inmutable
 - Acceso limitado al mercado secundario de equity privado
 - Onboarding lento (días) de nuevos inversionistas a una oferta
 - Falta de transparencia en titularidad y movimientos accionarios
 - Costos altos de mantener registros conforme a Disposiciones CNBV
 - Imposibilidad práctica de hacer "forced transfer" (recovery por pérdida de claves) en infraestructura tradicional
-

2. Qué construimos

- Plataforma web institucional con tres portales (Inversionista, Emisor IFC, Compliance Admin)
- Landing institucional con hero animado (shader cosmic + orbital architecture viz)

- Mercado secundario con orderbook animado (matching off-chain, settlement on-chain)
 - Sistema de identidad on-chain (ERC-3643 / T-REX) con módulos de compliance intercambiables
 - KYC orchestrator (mock, listo para integrar Truora/Mati/Sumsub)
 - Indexer event-driven que mantiene cap_table sincronizada
 - API REST regulada con auth real, RBAC y audit log
 - Mock blockchain SDK con la misma interfaz que tendrá la integración real (swap trivial)
 - Smart contracts (esqueleto) listos para deploy en Avalanche Fuji
 - Wallet integration con Core Wallet (Avalanche-native) priorizada
 - Dashboard con balances reales (AVAX/USDC) leídos en vivo del RPC de Fuji
 - Tabla de actividad on-chain real vía Routerscan API
 - Loading screen con shader animation (Three.js) entre login y dashboard
 - Editor de perfil con actualización vía PATCH /api/users/me
 - Connect-wallet gating: sin wallet conectada no se muestran datos sintéticos
-

3. Tecnologías y librerías

3.1 Frontend (apps/web)

- Next.js 15 (App Router, Turbopack)
- React 19
- TypeScript estricto (`noUncheckedIndexedAccess`)
- Tailwind CSS 3.4
- shadcn/ui
- Radix UI primitives: accordion, avatar, checkbox, dialog, dropdown-menu, label, popover, progress, radio-group, select, separator, slot, switch, tabs, tooltip
- Framer Motion 11
- Zustand 5 (estado cliente persistente)
- TanStack Query v5 (data fetching + cache)
- TanStack Table v8 (DataTable)
- React Hook Form 7 + @hookform/resolvers + Zod
- wagmi v2
- viem v2
- RainbowKit 2 (con Core Wallet, MetaMask, Coinbase, Rabby, Rainbow, Trust, Injected)
- Recharts 2 (charts)
- date-fns 4 (con locale es)
- lucide-react (iconos)
- cmdk (command palette ⌘K)
- class-variance-authority (variantes de componentes)

- clsx + tailwind-merge (composición de clases)
- Sonner (toasts)
- next-themes (preparado para light/dark)
- Three.js (shader animation en loading screen post-login)
- GSAP (animación cosmic del Hero canvas)

3.2 Backend (apps/web/src/app/api + apps/web/src/lib/server)

- Node.js 20
- Next.js Route Handlers (Promise-based params, Next 15 signature)
- Prisma ORM 5
- PostgreSQL 16 (vía Supabase)
- Redis 7 (vía Upstash, TLS)
- ioredis (client TCP)
- BullMQ 5 (queues + workers)
- jose (JWT HS256, cookie httpOnly)
- bcryptjs (password hashing)
- pino + pino-pretty (logging estructurado con redaction de PII)
- viem (recoverMessageAddress para SIWE-like flow)
- Zod (validación de inputs en boundary)
- dotenv-cli (carga `.env` del root del monorepo)

3.3 Indexer (apps/indexer)

- Node.js 20 ESM
- tsx (runtime + watch en dev)
- ioredis (consumer Redis Streams con consumer group)
- pino
- @prisma/client (writes a Postgres)

3.4 Smart contracts (packages/contracts) — preparado, no deployado

- Solidity 0.8.24 (viaIR)
- Hardhat 2.x
- @nomicfoundation/hardhat-toolbox
- OpenZeppelin contracts 5
- Estándar ERC-3643 (T-REX) como base
- ethers v6
- TypeChain
- hardhat-gas-reporter
- solidity-coverage

3.5 Mock blockchain SDK (packages/sdk)

- TypeScript puro (interfaces + implementaciones in-memory)
- EventBus tipado (eventemitter3-like, propio)
- Redis Streams para persistencia de eventos al indexer
- viem para EIP-712 typed data y formatos compatibles

3.6 Database package (packages/database)

- Prisma schema con 11 modelos
- Migrations versionadas
- Seed con datos demo realistas en español

3.7 Compartido (packages/shared)

- Zod schemas (DTOs)
- Tipos compartidos entre frontend y backend
- AppError jerarquía tipada (con `code` , `httpStatus` , `userMessage`)
- Constantes (roles, rate limits, jurisdicción MX = 484)

3.8 UI library (packages/ui)

- 35+ componentes propios sobre Radix + Tailwind (Button, Card, DataTable, OrderbookView, CommandPalette, MetricGrid, ChartCard, WalletAddress, Money, Sparkline, etc.)
- Design tokens en CSS vars (preparado para light/dark)
- CVA + cn utility

3.9 Infraestructura / DevOps

- pnpm 10 (workspaces)
- Turborepo 2 (task pipelines)
- Husky + lint-staged (git hooks)
- ESLint 9 + Prettier 3
- TypeScript 5.7
- Docker Compose (alternativa local para Postgres + Redis)

3.10 Servicios externos

- Supabase (Postgres hosted, free tier)
- Upstash (Redis Streams hosted, free tier)
- Routescan / Snowtrace API (lectura de transacciones reales en Fuji, sin API key)
- Avalanche Fuji testnet (RPC público)
- Pinata (IPFS, planeado para prospectos)
- Reown / WalletConnect Cloud (opcional, habilita conexión móvil con QR)

- Avalanche faucet (testnet AVAX)
 - GitHub (repo + planeado: GitHub Actions para CI)
 - Vercel (deploy planeado del web)
 - Railway (deploy planeado del indexer)
 - AvaCloud (planeado para subnet propia en producción)
-

4. Componentes del sistema

4.1 Smart contracts

- IdentityRegistry
- ClaimIssuer
- ComplianceRegistry
- HoldingPeriodModule
- MaxHoldersModule
- JurisdictionModule
- MaxInvestmentModule
- SecurityToken (ERC-3643)
- TokenFactory
- OrderBook
- Settlement
- Escrow
- MockUSDC

4.2 Endpoints API (22)

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- GET /api/auth/session
- GET /api/users/me
- PATCH /api/users/me
- POST /api/users/me/wallet (SIWE-like)
- POST /api/kyc/start
- POST /api/kyc/webhook (idempotente, HMAC)
- GET /api/offerings (paginado + filtros)
- GET /api/offerings/[id]
- POST /api/offerings (issuer/admin)
- GET /api/offerings/[id]/cap-table

- POST /api/orders (verifica EIP-712)
- GET /api/orders/book/[offeringId]
- DELETE /api/orders/[id]
- POST /api/match/execute
- GET /api/portfolio/[wallet]
- GET /api/trades
- POST /api/admin/freeze
- POST /api/admin/unfreeze
- POST /api/admin/whitelist
- GET /api/admin/audit-log
- GET /api/compliance/check

4.3 Modelos de base de datos (11)

- User
- Identity
- Wallet
- Issuer
- Offering
- CapTableEntry
- Order
- Trade
- KycRecord
- AuditLog
- Notification
- ProcessedEvent (para idempotencia del indexer)

4.4 Eventos del mock blockchain

- IdentityRegistered
- IdentityRemoved
- WalletFrozen
- WalletUnfrozen
- TokenDeployed
- Transfer
- ForcedTransfer
- OrderPosted
- OrderCancelled
- OrderFilled
- TradeExecuted

4.5 Páginas frontend (~30)

- Landing institucional
- /login, /register, /forgot-password
- /onboarding (4 pasos: datos, KYC mock, wallet, complete)
- /investor (dashboard, portfolio, offerings, offering detail, orders, trades, activity, watchlist, profile)
- /issuer (dashboard, offerings, offering detail, cap table, holders, analytics, new offering multi-step)
- /admin (dashboard, investors, compliance, jurisdictions, audit log)

4.6 Hooks y servicios cliente clave

- useSession (auth state)
- useWallet (wagmi wrap)
- useWalletBalances (AVAX + USDC reales)
- useFujiActivity (transacciones reales de Routescan)
- usePortfolio, usePortfolioHistory (mock derivado de wallet)
- useMyOrders, useMyTrades (mock derivado de wallet)
- useOrderbook (animated, simula realtime)

5. Decisiones arquitectónicas clave

- Subnet propia en Avalanche en lugar de C-Chain (validadores de la IFC, gas en stablecoin, permissioned)
- Matching off-chain + settlement on-chain (no orderbook 100% on-chain por costo y velocidad)
- ERC-3643 (T-REX) en lugar de ERC-1404 / ERC-1400
- Identity Registry separado del token (modular, evolucionable sin redeploy)
- Módulos de compliance intercambiables por configuración
- PostgreSQL en lugar de MongoDB (precisión decimal, transactions, fintech standard)
- Mock blockchain layer con la misma interfaz que el real (swap trivial post-deploy)
- jose para JWT en lugar de NextAuth (más liviano, no necesitamos OAuth providers)
- Cookie httpOnly para sesión (no localStorage)
- Datos del dashboard derivados de wallet conectada (no fakes globales)
- Connect-wallet gating: sin wallet conectada el dashboard no muestra datos sintéticos
- Trades/órdenes IFC en empty state honesto + actividad real Fuji vía Routescan
- pnpm workspaces (no npm/yarn) por velocidad y manejo de monorepo
- Turborepo para pipelines paralelos
- Supabase + Upstash hosted en lugar de Docker local (cero instalación, demoable desde cualquier máquina)
- Redis Streams para event bus (en lugar de Kafka/SNS para hackathon scope)
- Core Wallet priorizada en grupo "Avalanche" en el modal de RainbowKit

6. Flujos clave

- Onboarding de inversionista (registro con nombre + email/password → KYC mock → connect wallet → SIWE-like signature → identity registered on-chain)
 - Emisión de oferta (issuer crea oferta → upload prospecto → configura compliance modules → TokenFactory.deployToken → mint inicial)
 - Trade en mercado secundario (sign EIP-712 sell order → matching engine off-chain → Settlement.executeMatch atómico → indexer actualiza cap_table)
 - Compliance ops (freeze wallet, unfreeze, forced transfer por recovery, update whitelist, pause token)
 - Distribución de dividendos (planeado, no implementado)
 - Profile update (PATCH /api/users/me con firstName + lastName)
-

7. Compliance / Regulatorio

7.1 Marco legal

- Ley para Regular las Instituciones de Tecnología Financiera (Ley Fintech, 2018)
- Disposiciones de carácter general aplicables a las IFC (CNBV)
- Ley del Mercado de Valores (LMV)
- Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)

7.2 Conceptos clave

- Inversionista calificado vs no calificado (CNBV)
- Topes de inversión por persona y por oferta (Ley Fintech)
- Jurisdicción permitida (ISO 3166-1 numeric, MX = 484)
- Holding period / lockup obligatorio
- Max holders por oferta
- Audit log inmutable como requisito CNBV
- Forced transfer por recovery (pérdida de claves o herencia)
- Freeze wallet por orden judicial / sospecha AML
- KYC issuer y firma de claims
- Sandbox CNBV como ruta de adopción

7.3 Cómo se cumple (mapeo)

- Validación de jurisdicción → JurisdictionModule
- Tope inversionista no calificado → MaxInvestmentModule
- Lockup → HoldingPeriodModule

- Límite de inversionistas por oferta → MaxHoldersModule
 - Forced transfer / freeze → SecurityToken (onlyAgent)
 - Audit log → tabla append-only en Postgres + eventos on-chain
 - KYC trazable → ClaimIssuer firma claims, IdentityRegistry mapea wallet → identidad
-

8. Seguridad

- bcrypt para passwords (cost factor 10)
 - JWT firmado con jose (HS256), cookie httpOnly + Secure + SameSite
 - Rate limiting por endpoint (Redis sliding window)
 - Validación Zod en cada API boundary
 - Verificación de firma EIP-712 antes de aceptar órdenes
 - SIWE-like flow para vincular wallet a cuenta
 - PII redaction en logs (pino con redact paths)
 - RBAC: roles `investor` , `issuer` , `admin`
 - AppError tipado con códigos estables (no leak de stack traces)
 - Idempotencia en webhooks KYC (HMAC verify)
 - Audit log antes de ejecutar cualquier admin action
 - `noUncheckedIndexedAccess` en TS strict
 - Wallet privada del KYC issuer en env (nunca en código)
-

9. Estado actual del proyecto

9.1 Completado y funcional

- Monorepo con tooling (pnpm, Turbo, ESLint, Prettier, Husky)
- Frontend completo de los 3 portales (~30 páginas)
- Auth real (login, register, logout, session) con bcrypt + JWT
- Profile editor (update firstName/lastName via PATCH)
- DB schema migrada en Supabase con seed de datos demo
- Mock blockchain SDK con 5 adapters + EventBus
- Indexer event-driven funcional
- 22 endpoints API con validación + RBAC + rate limiting
- Wallet integration (Core, MetaMask, Coinbase, Rabby, Injected; WalletConnect opcional)
- Dashboard gateado por wallet conectada
- Balances reales AVAX/USDC desde RPC de Fuji
- Tabla de actividad real Fuji vía Routerscan

- Loading screen con shader Three.js
- Hero canvas con cosmic animation (GSAP)
- Architecture viz orbital (radial timeline interactivo)
- Logo del dashboard navega al portal correcto
- MarketingNav consciente de sesión

9.2 Mock o placeholder (no son datos reales)

- Holdings IFC en dashboard (derivados determinísticamente de wallet, no son tokens reales)
- Portfolio history (interpolación seeded por wallet)
- Mis órdenes IFC (vacío, sólo se muestra empty state honesto)
- Mis trades IFC (vacío, sólo empty state honesto)
- Orderbook animado (datos sintéticos por timer)
- KYC verification (UI mock, sin provider real)
- Distribución de dividendos
- Mapa de jurisdicciones (SVG simple, no topología real)

9.3 Pendiente

- Deploy real de smart contracts a Avalanche Fuji
- Tests unitarios de smart contracts (Hardhat)
- Auditoría de smart contracts
- Integración con KYC provider real (Truora / Mati / Sumsb)
- Workers BullMQ corriendo persistentes (hoy se invocan manualmente)
- WalletConnect mobile QR (requiere Reown project ID)
- Pitch deck final
- Video demo
- Subnet propia en AvaCloud (post-hackathon)
- Sandbox CNBV (post-hackathon)

10. Equipo y roles

(documentador: llenar con miembros del equipo)

- Nombre · rol · responsabilidades · contacto

11. Material a producir / Deliverables

- README pulido con pitch + quickstart + screenshots

- ARCHITECTURE.md actualizado al estado real
 - Pitch deck (10–12 slides)
 - Demo script (3 minutos)
 - Video demo (con audio limpio + subtítulos)
 - One-pager PDF
 - Live demo URL pública (Vercel)
 - Repo público en GitHub con tag de release
 - Diagrama ER de la base de datos
 - Diagramas de flujo (sequence diagrams) por flujo clave
-

12. URLs y recursos

- Repo: <https://github.com/Kaii35/hackathon-avalanche>
 - Demo local: <http://localhost:3000>
 - Indexer healthcheck local: <http://localhost:3001/health>
 - Avalanche Fuji RPC: <https://api.avax-test.network/ext/bc/C/rpc>
 - Snowtrace testnet: <https://testnet.snowtrace.io>
 - Avalanche faucet: <https://faucet.avax.network>
 - Reown / WalletConnect Cloud: <https://cloud.reown.com>
 - Supabase: <https://supabase.com>
 - Upstash: <https://console.upstash.com>
 - Routerscan API base: <https://api.routerscan.io/v2/network/testnet/evm/43113>
-

13. Cómo usar este documento

Para cada ítem de las secciones 1–12, el documentador debe escribir, en otro documento, **al menos**:

- **Qué es:** una definición corta y específica al proyecto
- **Por qué se eligió / por qué es importante:** trade-off considerado, alternativas descartadas
- **Para qué sirve dentro del producto:** dónde y cómo se usa
- **Estado:** producción-ready, mock, pendiente
- **Dónde vive en el repo:** ruta de archivo o carpeta cuando aplique
- **Quién lo necesita entender:** dev, jurado, IFC, auditor